TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 6: TÌM KIẾM CỤC BỘ & TỐI ƯU

Hill Climbing, Simulated Annealing và Giải thuật di truyền (GA)

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày:27/10/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả (kể cả hình biểu đồ) và dán vào khung viền xám đứt nét.
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên
- ⚠️ Lưu ý: Ngay sau Buổi 6 sẽ có bài kiểm tra module "Tìm kiếm & Tối ưu" (bao gồm nội dung Buổi 4–6) — hãy ôn tập kỹ các đoạn code đã cải tiến của bạn

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Phân biệt được không gian tìm kiếm liên tục với không gian trạng thái rời rạc đã học ở Buổi 4–5
- Giải thích được cực trị địa phương và cực trị toàn cục là gì
- Cài đặt được Hill Climbing và Simulated Annealing
- Giải thích được vì sao Simulated Annealing có thể thoát khỏi cực trị địa phương
- Áp dụng được Giải thuật di truyền (GA) có sẵn để giải bài toán cái túi

TIẾT 1 — TRẢI NGHIỆM: KẸT VÀ THOÁT KẸT

Mở notebook "Demo_Buoi6.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Hill Climbing bị kẹt ở cực trị địa phương

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.sin(x) + np.sin(x/3)

def hill_climbing(f, x0, buoc=0.1, so_lan=100):
    x = x0
```

```
    duong_di = [x]
    for _ in range(so_lan):
        x_toi = x + buoc
        x_lui = x - buoc
        if f(x_toi) > f(x):
            x = x_toi
        elif f(x_lui) > f(x):
            x = x_lui
        else:
            break   # không còn cải thiện được -> dừng lại (có thể là cực trị
địa phương)
        duong_di.append(x)
    return x, f(x), duong_di

x_bat_dau = 2.0
x_ket_qua, gia_tri, duong_di = hill_climbing(f, x_bat_dau)
print("Bắt đầu tại x =", x_bat_dau)
print("Hill Climbing dừng tại x =", x_ket_qua, "với f(x) =", gia_tri)

xs = np.linspace(0, 20, 400)
plt.plot(xs, f(xs), label="f(x)")
plt.plot(duong_di, [f(x) for x in duong_di], "ro-", label="Đường đi Hill
Climbing")
plt.legend()
plt.title("Hill Climbing bị kẹt ở cực trị địa phương")
plt.show()
```

💭 **Suy nghĩ: Nhìn vào đồ thị f(x), điểm Hill Climbing dừng lại có phải là đỉnh cao nhất trong toàn bộ đồ thị không?**

Không. Điểm dừng thường chỉ là một đỉnh cục bộ (cực trị địa phương), không phải đỉnh cao nhất toàn đồ thị. Tùy điểm xuất phát mà nó kẹt ở các đỉnh khác nhau.

🔧 Cải tiến: Hãy đổi x_bat_dau sang 3–4 giá trị khác nhau (VD: 8.0, 14.0, 17.0), chạy lại và xem Hill Climbing dừng ở đâu mỗi lần:

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.sin(x) + np.sin(x/3)

def hill_climbing(f, x0, buoc=0.1, so_lan=100):
    x = x0
    duong_di = [x]
    for _ in range(so_lan):
        x_toi = x + buoc
        x_lui = x - buoc
        if f(x_toi) > f(x):
            x = x_toi
        elif f(x_lui) > f(x):
            x = x_lui
        else:
            break
        duong_di.append(x)
    return x, f(x), duong_di

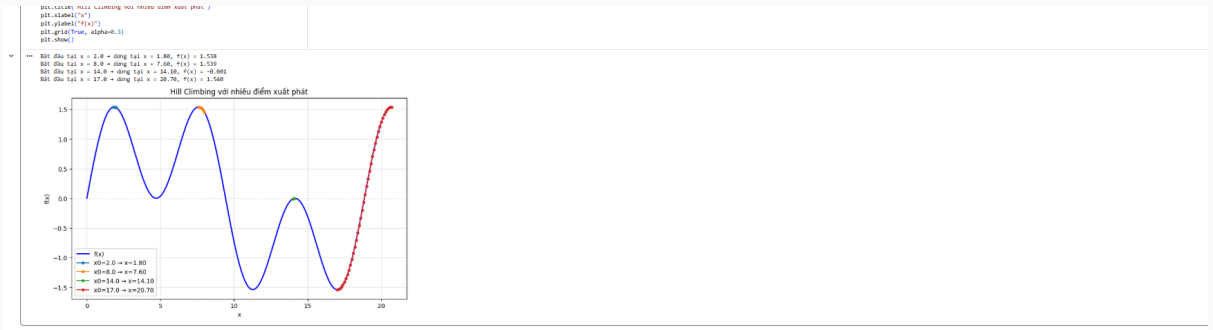
cac_diem_bat_dau = [2.0, 8.0, 14.0, 17.0]
xs = np.linspace(0, 20, 400)

plt.figure(figsize=(10, 6))
plt.plot(xs, f(xs), 'b-', linewidth=2, label='f(x)')
```

```
for x0 in cac_diem_bat_dau:
    x_kq, gia_tri, duong_di = hill_climbing(f, x0)
    plt.plot(duong_di, [f(x) for x in duong_di], 'o-', markersize=4, label=f'x0={x0} → x={x_kq:.2f}')
    print(f"Bắt đầu tại x = {x0} → dừng tại x = {x_kq:.2f}, f(x) = {gia_tri:.3f}")

plt.legend()
plt.title("Hill Climbing với nhiều điểm xuất phát")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.grid(True, alpha=0.3)
plt.show()
```

🌾 Thu hoạch: chụp màn hình đồ thị Hill Climbing với điểm bắt đầu mới:



Demo 2: Simulated Annealing thoát khỏi cực trị địa phương

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random
import math

def simulated_annealing(f, x0, nhiet_do_bd=10, toc_do_giam=0.95, so_lan=200):
    x = x0
    nhiet_do = nhiet_do_bd
    duong_di = [x]
    for _ in range(so_lan):
        x_moi = x + random.uniform(-1, 1)
        delta = f(x_moi) - f(x)
        if delta > 0 or random.random() < math.exp(delta / nhiet_do):
            x = x_moi
        nhiet_do *= toc_do_giam
        duong_di.append(x)
    return x, f(x), duong_di

x_ket_qua_sa, gia_tri_sa, duong_di_sa = simulated_annealing(f, x_bat_dau)
print("Simulated Annealing kết thúc tại x =", x_ket_qua_sa, "với f(x) =",
gia_tri_sa)

plt.plot(xs, f(xs), label="f(x)")
plt.plot(duong_di_sa, [f(x) for x in duong_di_sa], "go-", markersize=3,
label="Đường đi Simulated Annealing")
plt.legend()
plt.title("Simulated Annealing thoát khỏi cực trị địa phương")
plt.show()
```

💭 **Suy nghĩ: So với Hill Climbing (Demo 1) cùng điểm xuất phát x_bat_dau, Simulated Annealing có tìm được điểm tốt hơn không?**

Thường là có. Simulated Annealing thường tìm được điểm tốt hơn (gần cực trị toàn cục hơn) vì nó có thể nhảy ra khỏi vùng kẹt.

🔧 Cải tiến: Hãy dùng cùng x_bat_dau mà bạn đã đổi ở Demo 1, chạy Simulated Annealing và so sánh 2 kết quả:

```
import numpy as np
import matplotlib.pyplot as plt
import random
import math

def f(x):
    return np.sin(x) + np.sin(x/3)

def simulated_annealing(f, x0, nhiet_do_bd=10, toc_do_giam=0.95, so_lan=200):
    x = x0
    nhiet_do = nhiet_do_bd
    duong_di = [x]
    for _ in range(so_lan):
        x_moi = x + random.uniform(-1, 1)
        delta = f(x_moi) - f(x)
        if delta > 0 or random.random() < math.exp(delta / max(nhiet_do, 1e-9)):
            x = x_moi
        nhiet_do *= toc_do_giam
        duong_di.append(x)
    return x, f(x), duong_di

cac_diem_bat_dau = [2.0, 8.0, 14.0, 17.0]   # cùng với Demo 1
xs = np.linspace(0, 20, 400)

plt.figure(figsize=(10, 6))
plt.plot(xs, f(xs), 'b-', linewidth=2, label='f(x)')

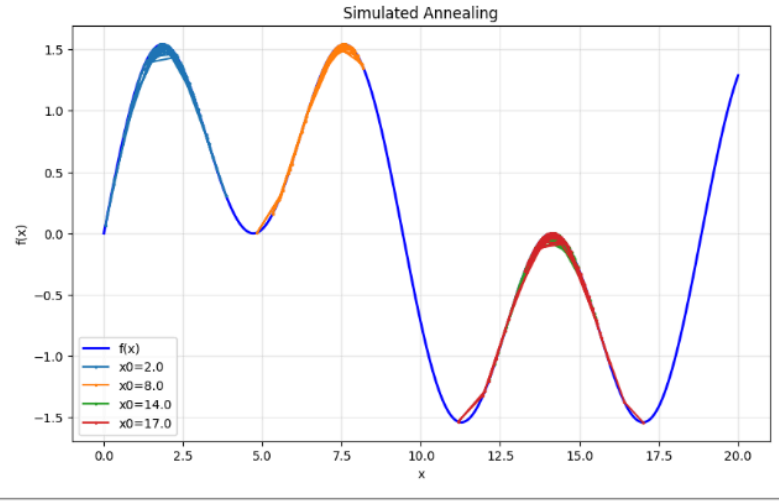
for x0 in cac_diem_bat_dau:
    x_kq, gia_tri, duong_di = simulated_annealing(f, x0)
    plt.plot(duong_di, [f(x) for x in duong_di], '.-', markersize=3, label=f'x0={x0} → x={x_kq:.2f}')
    print(f"SA bắt đầu tại x = {x0} → kết thúc tại x = {x_kq:.2f}, f(x) = {gia_tri:.3f}")

plt.legend()
plt.title("Simulated Annealing với nhiều điểm xuất phát")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.grid(True, alpha=0.3)
plt.show()
```

🌾 Thu hoạch: chụp màn hình đồ thị Simulated Annealing để so sánh với Hill Climbing:

```
plt.title("Simulated Annealing")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.grid(True, alpha=0.3)
plt.show()
```

```
x0 = 2.0 -> x = 1.82 , f = 1.539
x0 = 8.0 -> x = 7.58 , f = 1.54
x0 = 14.0 -> x = 14.17 , f = -0.001
x0 = 17.0 -> x = 14.1 , f = -0.001
```



Câu hỏi thảo luận: Vì sao Simulated Annealing đôi khi chấp nhận di chuyển đến điểm có f(x) THẤP HƠN? Điều đó có vẻ 'ngu ngốc' nhưng lại giúp ích gì?

Vì khi nhiệt độ còn cao, nó cho phép nhận bước xấu với xác suất exp(delta / nhiệt độ). Việc này giúp thoát khỏi cực trị địa phương để khám phá vùng khác. Ban đầu thì “ngu ngốc”, nhưng về lâu dài giúp tìm được lời giải tốt hơn thay vì bị kẹt sớm.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Không gian tìm kiếm liên tục khác không gian trạng thái rời rạc (mê cung, đồ thị) như thế nào?

Liên tục là các giá trị thực (ví dụ x trên trục số, đồ thị hàm liên tục). Rời rạc là các trạng thái riêng biệt (ô mê cung, đỉnh đồ thị, cấu hình N-Hậu…). Trong liên tục ta di chuyển bằng bước nhỏ, còn rời rạc thì nhảy từ trạng thái này sang trạng thái kề.

Cực trị địa phương và cực trị toàn cục khác nhau như thế nào?

Cực trị địa phương là điểm tốt nhất trong vùng lân cận của nó (hàng xóm không tốt hơn). Cực trị toàn cục là điểm tốt nhất trên toàn bộ không gian tìm kiếm. Hill Climbing dễ kẹt ở địa phương, còn SA/GA có cơ chế thoát ra để tìm toàn cục hơn.

Cơ chế 'nhiệt độ' trong Simulated Annealing hoạt động như thế nào? Vì sao nhiệt độ giảm dần theo thời gian?

Nhiệt độ cao → dễ chấp nhận bước xấu (khám phá nhiều). Nhiệt độ thấp → gần như chỉ nhận bước tốt (khai thác). Giảm dần theo thời gian để ban đầu tìm rộng, sau đó hội tụ về vùng tốt.

Giải thuật di truyền (GA) lấy cảm hứng từ đâu? Quần thể, lai ghép, đột biến trong GA nghĩa là gì?

Lấy cảm hứng từ chọn lọc tự nhiên. Quần thể là nhiều cá thể (lời giải) cùng tồn tại. Lai ghép (crossover) là kết hợp gen từ hai cha mẹ. Đột biến là thay đổi ngẫu nhiên một phần gen để tạo đa dạng.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: ĐIỀU CHỈNH THAM SỐ

📋 Nội dung có sẵn — Hill Climbing với nhiều điểm xuất phát (tái sử dụng hàm hill_climbing ở Tiết 1). Copy và chạy:

```
diem_bat_dau_list = [1, 5, 10, 15, 18]
ket_qua_hc = []
for x0 in diem_bat_dau_list:
    x_kq, gia_tri, _ = hill_climbing(f, x0)
    ket_qua_hc.append((x0, x_kq, gia_tri))
    print(f"Bắt đầu tại x={x0} -> dừng tại x={x_kq:.2f}, f(x)={gia_tri:.2f}")

x_tot_nhat = max(ket_qua_hc, key=lambda t: t[2])
print("Kết quả tốt nhất trong các lần chạy:", x_tot_nhat)
```

💭 **Suy nghĩ: Trong 5 điểm xuất phát, có bao nhiêu điểm cho ra kết quả khác nhau? Điều đó nói lên điều gì về Hill Climbing?**

Thường có 2–3 kết quả khác nhau (hoặc nhiều hơn tùy hàm). Điều này cho thấy Hill Climbing phụ thuộc mạnh vào điểm bắt đầu và dễ kẹt ở các cực trị địa phương khác nhau.

🔧 Cải tiến: Hãy thêm ít nhất 3 điểm xuất phát mới vào diem_bat_dau_list và xem kết quả tốt nhất có thay đổi không:

```
import numpy as np

def f(x):
    return np.sin(x) + np.sin(x/3)

def hill_climbing(f, x0, buoc=0.1, so_lan=100):
    x = x0
    for _ in range(so_lan):
        x_toi = x + buoc
        x_lui = x - buoc
        if f(x_toi) > f(x):
            x = x_toi
        elif f(x_lui) > f(x):
            x = x_lui
        else:
            break
    return x, f(x)

diem_bat_dau_list = [1, 3, 5, 7, 10, 12, 15, 16.5, 18, 19]
ket_qua = []

print("=== Hill Climbing đa điểm xuất phát ===")
for x0 in diem_bat_dau_list:
    x_kq, gia_tri = hill_climbing(f, x0)
    ket_qua.append((x0, x_kq, gia_tri))
    print(f"x0={x0:5} → dừng tại x={x_kq:.2f}, f(x)={gia_tri:.3f}")

tot_nhat = max(ket_qua, key=lambda t: t[2])
print(f"\nKết quả tốt nhất: bắt đầu x={tot_nhat[0]}, dừng x={tot_nhat[1]:.2f}, f={tot_nhat[2]:.3f}")
```

🌾 Thu hoạch: chụp màn hình kết quả chạy với danh sách điểm xuất phát mới:

```python
import numpy as np

def f(x):
    return np.sin(x) + np.sin(x/3)

def hill_climbing(f, x0, buoc=0.1, so_lan=100):
    x = x0
    for _ in range(so_lan):
        x_toi = x + buoc
        x_lui = x - buoc
        if f(x_toi) > f(x):
            x = x_toi
        elif f(x_lui) > f(x):
            x = x_lui
        else:
            break
    return x, f(x)

diem_bat_dau_list = [1, 3, 5, 7, 10, 12, 15, 16.5, 18, 19]
ket_qua = []

print("=== Hill Climbing đa điểm xuất phát ===")
for x0 in diem_bat_dau_list:
    x_kq, gia_tri = hill_climbing(f, x0)
    ket_qua.append((x0, x_kq, gia_tri))
    print(f"x0={x0:5} → dừng tại x={x_kq:.2f}, f(x)={gia_tri:.3f}")

tot_nhat = max(ket_qua, key=lambda t: t[2])
print(f"\nKết quả tốt nhất: bắt đầu x={tot_nhat[0]}, dừng x={tot_nhat[1]:.2f}, f={tot_nhat[2]:.3f}")
```

```
=== Hill Climbing đa điểm xuất phát ===
x0=    1 → dừng tại x=1.80, f(x)=1.538
x0=    3 → dừng tại x=1.80, f(x)=1.538
x0=    5 → dừng tại x=7.60, f(x)=1.539
x0=    7 → dừng tại x=7.60, f(x)=1.539
x0=   10 → dừng tại x=7.60, f(x)=1.539
x0=   12 → dừng tại x=14.10, f(x)=-0.001
x0=   15 → dừng tại x=14.10, f(x)=-0.001
x0= 16.5 → dừng tại x=14.10, f(x)=-0.001
x0=   18 → dừng tại x=20.70, f(x)=1.540
x0=   19 → dừng tại x=20.70, f(x)=1.540

Kết quả tốt nhất: bắt đầu x=19, dừng x=20.70, f=1.540
```

Cải tiến: thêm ít nhất 3 điểm xuất phát mới:

📋 Nội dung có sẵn — điều chỉnh tham số Simulated Annealing (tái sử dụng hàm simulated_annealing ở Tiết 1). Copy và chạy:

```
for nhiet_do_bd in [1, 10, 50]:
    x_kq, gia_tri, _ = simulated_annealing(f, x0=2.0, nhiet_do_bd=nhiet_do_bd)
        print(f"Nhiệt độ ban đầu={nhiet_do_bd} -> dừng tại x={x_kq:.2f}, f(x)={gia_tri:.2f}")
```

💭 **Suy nghĩ: Nhiệt độ ban đầu càng cao thì kết quả có xu hướng thay đổi như thế nào?**

Nhiệt độ ban đầu cao thường cho kết quả đa dạng hơn, dễ thoát cục bộ hơn, nhưng nếu quá cao thì có thể dao động nhiều và hội tụ chậm. Nhiệt độ thấp thì gần giống Hill Climbing.

🔧 Cải tiến: Hãy thử thêm các giá trị toc_do_giam khác nhau (VD: 0.80, 0.90, 0.99) với nhiệt độ ban đầu cố định, quan sát tốc độ giảm nhiệt ảnh hưởng đến kết quả ra sao:

```
import numpy as np
import random
import math

def f(x):
    return np.sin(x) + np.sin(x/3)

def simulated_annealing(f, x0, nhiet_do_bd=10, toc_do_giam=0.95, so_lan=300):
    x = x0
    nhiet_do = nhiet_do_bd
    for _ in range(so_lan):
        x_moi = x + random.uniform(-1, 1)
        delta = f(x_moi) - f(x)
        if delta > 0 or random.random() < math.exp(delta / max(nhiet_do, 1e-9)):
            x = x_moi
        nhiet_do *= toc_do_giam
    return x, f(x)
```

```
print("=== Thử các tốc độ giảm nhiệt (x0=2.0, nhiệt độ ban đầu=10) ===")
for toc in [0.80, 0.90, 0.95, 0.99]:
    x_kq, gia_tri = simulated_annealing(f, 2.0, toc_do_giam=toc)
    print(f"toc_do_giam = {toc} → x = {x_kq:.2f}, f(x) = {gia_tri:.3f}")
```

🌾 Thu hoạch: chụp màn hình kết quả với các giá trị toc_do_giam khác nhau:

```
import numpy as np
import random
import math

def f(x):
    return np.sin(x) + np.sin(x/3)

def simulated_annealing(f, x0, nhiet_do_bd=10, toc_do_giam=0.95, so_lan=300):
    x = x0
    nhiet_do = nhiet_do_bd
    for _ in range(so_lan):
        x_moi = x + random.uniform(-1, 1)
        delta = f(x_moi) - f(x)
        if delta > 0 or random.random() < math.exp(delta / max(nhiet_do, 1e-9)):
            x = x_moi
        nhiet_do *= toc_do_giam
    return x, f(x)

print("=== Thử các tốc độ giảm nhiệt (x0=2.0, nhiệt độ ban đầu=10) ===")
for toc in [0.80, 0.90, 0.95, 0.99]:
    x_kq, gia_tri = simulated_annealing(f, 2.0, toc_do_giam=toc)
    print(f"toc_do_giam = {toc} → x = {x_kq:.2f}, f(x) = {gia_tri:.3f}")
```

```
=== Thử các tốc độ giảm nhiệt (x0=2.0, nhiệt độ ban đầu=10) ===
toc_do_giam = 0.8 → x = 1.85, f(x) = 1.540
toc_do_giam = 0.9 → x = 1.85, f(x) = 1.540
toc_do_giam = 0.95 → x = 1.85, f(x) = 1.540
toc_do_giam = 0.99 → x = 1.11, f(x) = 1.260
```

TIẾT 4 — ÁP DỤNG MỞ RỘNG: GIẢI THUẬT DI TRUYỀN CHO BÀI TOÁN CÁI TÚI

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy code Giải thuật di truyền (GA) giải bài toán cái túi sau:

```
import random

# Bài toán cái túi: mỗi vật có trọng lượng và giá trị, túi chứa tối đa 15kg
trong_luong = [2, 3, 4, 5, 9, 7]
gia_tri =     [3, 4, 5, 8, 10, 6]
suc_chua = 15

def tinh_fitness(ca_the):
    tong_trong_luong = sum(t*c for t, c in zip(trong_luong, ca_the))
    tong_gia_tri = sum(g*c for g, c in zip(gia_tri, ca_the))
    if tong_trong_luong > suc_chua:
        return 0   # vi phạm ràng buộc -> fitness = 0
    return tong_gia_tri

def tao_ca_the_ngau_nhien():
    return [random.randint(0, 1) for _ in range(len(trong_luong))]
```

```
def lai_ghep(cha, me):
    diem_cat = random.randint(1, len(cha)-1)
    return cha[:diem_cat] + me[diem_cat:]

def dot_bien(ca_the, xac_suat=0.1):
    return [1-gene if random.random() < xac_suat else gene for gene in ca_the]

def giai_thuat_di_truyen(so_the_he=50, kich_thuoc_quan_the=20):
    quan_the = [tao_ca_the_ngau_nhien() for _ in range(kich_thuoc_quan_the)]
    for the_he in range(so_the_he):
        quan_the.sort(key=tinh_fitness, reverse=True)
        the_he_moi = quan_the[:4]   # giữ lại 4 cá thể tốt nhất (elitism)
        while len(the_he_moi) < kich_thuoc_quan_the:
            cha, me = random.choices(quan_the[:10], k=2)
            con = dot_bien(lai_ghep(cha, me))
            the_he_moi.append(con)
        quan_the = the_he_moi
    quan_the.sort(key=tinh_fitness, reverse=True)
    return quan_the[0], tinh_fitness(quan_the[0])

ca_the_tot_nhat, fitness_tot_nhat = giai_thuat_di_truyen()
print("Cá thể tốt nhất (0/1 cho từng vật):", ca_the_tot_nhat)
print("Tổng giá trị đạt được:", fitness_tot_nhat)
```

☐ Chạy code trên và xác định vật nào được chọn dựa vào cá thể tốt nhất (giá trị 1 = chọn, 0 = không chọn)

🔧 Cải tiến: Cả nhóm cùng thêm ít nhất 2 vật phẩm mới vào trong_luong/gia_tri, đổi suc_chua, chạy lại và quan sát cá thể tốt nhất thay đổi ra sao; sau đó thử đổi so_the_he (VD: 10 vs 100) để xem số thế hệ ảnh hưởng đến chất lượng lời giải như thế nào:

```
import random

# Cải tiến: thêm 2 vật phẩm mới + đổi sức chứa
trong_luong = [2, 3, 4, 5, 9, 7, 6, 8]     # thêm 2 món
gia_tri     = [3, 4, 5, 8, 10, 6, 9, 7]    # giá trị tương ứng
suc_chua    = 20                    # tăng sức chứa

def tinh_fitness(ca_the):
    tong_trong_luong = sum(t*c for t, c in zip(trong_luong, ca_the))
    tong_gia_tri     = sum(g*c for g, c in zip(gia_tri, ca_the))
    if tong_trong_luong > suc_chua:
        return 0
    return tong_gia_tri

def tao_ca_the_ngau_nhien():
    return [random.randint(0, 1) for _ in range(len(trong_luong))]

def lai_ghep(cha, me):
    diem_cat = random.randint(1, len(cha)-1)
    return cha[:diem_cat] + me[diem_cat:]

def dot_bien(ca_the, xac_suat=0.1):
    return [1-gene if random.random() < xac_suat else gene for gene in ca_the]

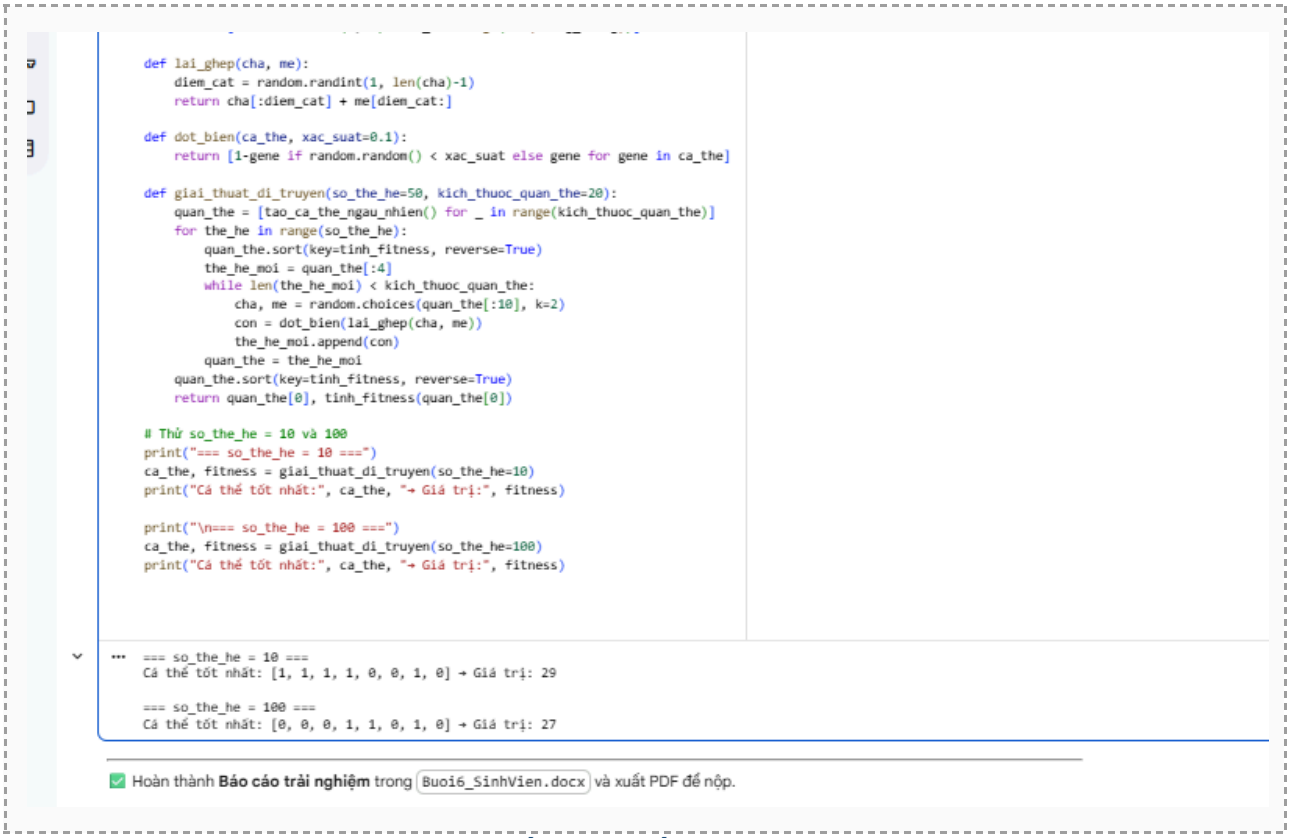
def giai_thuat_di_truyen(so_the_he=50, kich_thuoc_quan_the=20):
    quan_the = [tao_ca_the_ngau_nhien() for _ in range(kich_thuoc_quan_the)]
    for the_he in range(so_the_he):
        quan_the.sort(key=tinh_fitness, reverse=True)
        the_he_moi = quan_the[:4]
        while len(the_he_moi) < kich_thuoc_quan_the:
```

```
        cha, me = random.choices(quan_the[:10], k=2)
        con = dot_bien(lai_ghep(cha, me))
        the_he_moi.append(con)
    quan_the = the_he_moi
quan_the.sort(key=tinh_fitness, reverse=True)
return quan_the[0], tinh_fitness(quan_the[0])

# Thử so_the_he = 10 và 100
print("=== so_the_he = 10 ===")
ca_the, fitness = giai_thuat_di_truyen(so_the_he=10)
print("Cá thể tốt nhất:", ca_the, "→ Giá trị:", fitness)

print("\n=== so_the_he = 100 ===")
ca_the, fitness = giai_thuat_di_truyen(so_the_he=100)
print("Cá thể tốt nhất:", ca_the, "→ Giá trị:", fitness)
```

🌾 Thu hoạch: chụp màn hình kết quả cá thể tốt nhất sau khi cải tiến:



Nhận xét của nhóm: khi tăng so_the_he, kết quả có tốt hơn không? Vì sao?

Thường tốt hơn (fitness cao hơn hoặc ổn định hơn). Vì có thêm nhiều thế hệ để chọn lọc, lai ghép và đột biến, quần thể có thời gian tiến hóa đến vùng lời giải tốt hơn. Nhưng quá nhiều thế hệ thì lợi ích giảm dần và tốn thời gian.

BÀI THỰC HÀNH DÀNH CHO SINH VIÊN CNTT

Năm bài dưới đây dùng đúng các kỹ thuật của Buổi 6 (Hill Climbing, Random Restart, Simulated Annealing, Giải thuật di truyền) trên những bài toán quen thuộc của ngành CNTT. Mỗi bài đều có code chạy sẵn trong notebook Thuchanh_Buoi6.ipynb: bạn chỉ cần copy/paste → chạy → suy nghĩ → cải tiến → thu hoạch. Các bài độc lập, có thể làm theo thứ tự bất kỳ (Bài 4 chạy khoảng 15–20 giây).

Bài 1 — Bài toán N quân Hậu bằng Hill Climbing & Random Restart

Liên hệ CNTT: N-Queens là bài toán thỏa mãn ràng buộc (CSP) kinh điển, cùng họ với bài toán SAT, xếp lịch, cấu hình hệ thống. Kỹ thuật "đếm số ràng buộc bị vi phạm rồi sửa dần" (min-conflicts) được dùng thật trong các bộ giải CSP.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random

def so_xung_dot(tt):
	"""tt[i] = hàng của quân hậu đặt ở cột i. Trả về số cặp hậu đang chiếu nhau."""
	n = len(tt)
	dem = 0
	for i in range(n):
		for j in range(i + 1, n):
			if tt[i] == tt[j] or abs(tt[i] - tt[j]) == j - i:
			dem += 1
	return dem

def hill_climbing_hau(n=8, toi_da_buoc=200):
	tt = [random.randint(0, n - 1) for _ in range(n)]		# trạng thái ngẫu nhiên
	for _ in range(toi_da_buoc):
	hien_tai = so_xung_dot(tt)
	if hien_tai == 0:
		return tt, 0
	tot_nhat, tt_tot_nhat = hien_tai, None
	for cot in range(n):					# thử dời từng quân hậu
		for hang in range(n):
		if hang != tt[cot]:
			moi = tt[:]
			moi[cot] = hang
			xd = so_xung_dot(moi)
			if xd < tot_nhat:
			tot_nhat, tt_tot_nhat = xd, moi
	if tt_tot_nhat is None:					# không bước nào tốt hơn -> kẹt
		return tt, hien_tai
	tt = tt_tot_nhat
	return tt, so_xung_dot(tt)

def hc_khoi_dong_lai(n=8, toi_da_lan=100):
	for lan in range(1, toi_da_lan + 1):
	tt, xd = hill_climbing_hau(n)
	if xd == 0:
		return tt, lan
	return None, toi_da_lan

def in_ban_co(tt):
	n = len(tt)
	for hang in range(n):
	print(" ".join("Q" if tt[cot] == hang else "." for cot in range(n)))

random.seed(42)
so_lan_chay = 200
thanh_cong = sum(1 for _ in range(so_lan_chay) if hill_climbing_hau(8)[1] == 0)
print(f"Hill Climbing thuần: giải được {thanh_cong}/{so_lan_chay} lần
({100*thanh_cong/so_lan_chay:.0f}%)")

so_lan_khoi_dong = []
for _ in range(100):
	_, lan = hc_khoi_dong_lai(8)
	so_lan_khoi_dong.append(lan)
print(f"Random Restart: trung bình cần {sum(so_lan_khoi_dong)/len(so_lan_khoi_dong):.1f} lần khởi
động để ra lời giải")

loi_giai, lan = hc_khoi_dong_lai(8)
print(f"\nMột lời giải 8 hậu (tìm được sau {lan} lần khởi động):")
in_ban_co(loi_giai)
```

```
so_lan_khoi_dong = []
for _ in range(100):
    _, lan = hc_khoi_dong_lai(8)
    so_lan_khoi_dong.append(lan)
print(f"Random Restart: trung bình cần {sum(so_lan_khoi_dong)/len(so_lan_khoi_dong):.1f} lần khởi động để ra lời giải")

loi_giai, lan = hc_khoi_dong_lai(8)
print(f"\nMột lời giải 8 hậu (tìm được sau {lan} lần khởi động):")
in_ban_co(loi_giai)
```

```
Hill Climbing thuần: giải được 32/200 lần (16%)
Random Restart: trung bình cần 6.5 lần khởi động để ra lời giải

Một lời giải 8 hậu (tìm được sau 1 lần khởi động):
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
Q . . . . . . .
. . . . . Q . .
. . . . Q . . .
```

💭 **Suy nghĩ: Vì sao Hill Climbing thuần chỉ giải được một tỷ lệ nhỏ các lần chạy? Điều gì xảy ra ở những lần "thất bại" (gợi ý: so_xung_dot > 0 nhưng không có bước dời nào làm nó giảm)? Vì sao khởi động lại ngẫu nhiên lại khắc phục được?**

Hill Climbing thuần chỉ giải được tỷ lệ nhỏ vì nhiều lần nó kẹt ở trạng thái còn xung đột nhưng không còn bước dời nào làm giảm số xung đột (cực trị địa phương). Khởi động lại ngẫu nhiên khắc phục bằng cách thử nhiều điểm xuất phát khác nhau, tăng cơ hội tìm được vùng có lời giải.

🔧 Cải tiến: Đổi n = 8 thành n = 12, chạy lại và ghi nhận: tỷ lệ thành công của Hill Climbing thuần và số lần khởi động trung bình thay đổi thế nào khi bàn cờ lớn hơn? (Có thể thử cả n = 16, nhưng hãy giảm so_lan_chay xuống 30 và vòng lặp range(100) xuống 20 vì sẽ chạy khá chậm.)

```
# Chỉ cần đổi n=12 trong các lệnh gọi (giữ nguyên các hàm so_xung_dot, hill_climbing_hau, hc_khoi_dong_lai)


random.seed(42)

so_lan_chay = 100  # giảm bớt vì n=12 chậm hơn

thanh_cong = sum(1 for _ in range(so_lan_chay) if hill_climbing_hau(12)[1] == 0)

print(f"Hill Climbing thuần n=12: giải được {thanh_cong}/{so_lan_chay} lần ({100*thanh_cong/so_lan_chay:.0f}%)")


so_lan_khoi_dong = []

for _ in range(50):

  _, lan = hc_khoi_dong_lai(12, toi_da_lan=200)

  so_lan_khoi_dong.append(lan)

print(f"Random Restart n=12: trung bình cần {sum(so_lan_khoi_dong)/len(so_lan_khoi_dong):.1f} lần khởi động")
```

```
loi_giai, lan = hc_khoi_dong_lai(12)

print(f"\nMột lời giải 12 hậu (sau {lan} lần khởi động):")

in_ban_co(loi_giai)
```

```
loi_giai, lan = hc_khoi_dong_lai(12)
print(f"\nMột lời giải 12 hậu (sau {lan} lần khởi động):")
in_ban_co(loi_giai)
```

```
Hill Climbing thuần n=12: giải được 4/100 lần (4%)
Random Restart n=12: trung bình cần 21.1 lần khởi động

Một lời giải 12 hậu (sau 9 lần khởi động):
. . . Q . . . . . . . .
Q . . . . . . . . . . .
. . . . . Q . . . . . .
. . . . . . . . . Q . .
. Q . . . . . . . . . .
. . . . . Q . . . . . .
. . . . . . . . . . . Q
. . . . . . . . Q . . .
. . . . . . . . . . Q .
. . . . . . . Q . . . .
. . . . Q . . . . . . .
. . Q . . . . . . . . .
```

Bài 2 — Người bán hàng (TSP): định tuyến qua các máy chủ bằng Simulated Annealing

Liên hệ CNTT: TSP là mô hình của bài toán định tuyến gói tin/xe giao hàng, thiết kế mạch in, khoan lỗ PCB, lập lịch đọc đĩa. Ở đây một "nhân viên kỹ thuật" phải ghé 25 phòng máy chủ rồi quay về điểm xuất phát với quãng đường ngắn nhất.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random, math
import matplotlib.pyplot as plt

random.seed(1)
N = 25
phong_may = [(random.uniform(0, 100), random.uniform(0, 100)) for _ in range(N)]

def do_dai_tour(tour):
	return sum(math.dist(phong_may[tour[i]], phong_may[tour[(i + 1) % N]]) for i in range(N))

def lang_gieng_gan_nhat(bat_dau=0):
	chua_di = set(range(N)) - {bat_dau}
	tour = [bat_dau]
	while chua_di:
	gan_nhat = min(chua_di, key=lambda c: math.dist(phong_may[tour[-1]], phong_may[c]))
	tour.append(gan_nhat)
	chua_di.remove(gan_nhat)
	return tour

def dao_doan(tour, i, j):
	"""Thao tác 2-opt: đảo ngược đoạn tour[i..j]."""
	return tour[:i] + tour[i:j + 1][::-1] + tour[j + 1:]

def hill_climbing_tsp(tour, so_lan=30000):
	tot_nhat = do_dai_tour(tour)
	for _ in range(so_lan):
	i, j = sorted(random.sample(range(N), 2))
	moi = dao_doan(tour, i, j)
	d = do_dai_tour(moi)
	if d < tot_nhat:                    # chỉ nhận nếu tốt hơn
		tour, tot_nhat = moi, d
	return tour

def simulated_annealing_tsp(tour, nhiet_do=30.0, toc_do_giam=0.9998, so_lan=30000):
	hien_tai = do_dai_tour(tour)
	tour_tot_nhat, tot_nhat = tour, hien_tai
	for _ in range(so_lan):
	i, j = sorted(random.sample(range(N), 2))
	moi = dao_doan(tour, i, j)
	d = do_dai_tour(moi)
	delta = d - hien_tai
	if delta < 0 or random.random() < math.exp(-delta / nhiet_do):   # đôi khi nhận bước xấu
hơn
		tour, hien_tai = moi, d
		if hien_tai < tot_nhat:
		tour_tot_nhat, tot_nhat = tour, hien_tai
	nhiet_do *= toc_do_giam
	return tour_tot_nhat

tour_ngau_nhien = list(range(N))
random.shuffle(tour_ngau_nhien)
tour_nn = lang_gieng_gan_nhat()
tour_hc = hill_climbing_tsp(tour_ngau_nhien[:])           # HC và SA cùng xuất phát từ một tour
ngẫu nhiên
tour_sa = simulated_annealing_tsp(tour_ngau_nhien[:])

ket_qua = {"Ngẫu nhiên": tour_ngau_nhien, "Láng giềng gần nhất": tour_nn,
	"Hill Climbing (2-opt)": tour_hc, "Simulated Annealing": tour_sa}
for ten, t in ket_qua.items():
	print(f"{ten:<24}: độ dài = {do_dai_tour(t):7.1f}")

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (ten, t) in zip(axes, [("Láng giềng gần nhất", tour_nn), ("Hill Climbing (2-opt)",
tour_hc), ("Simulated Annealing", tour_sa)]):
	xs = [phong_may[c][0] for c in t + [t[0]]]
	ys = [phong_may[c][1] for c in t + [t[0]]]
	ax.plot(xs, ys, "o-")
	ax.set_title(f"{ten}\nđộ dài = {do_dai_tour(t):.1f}")
plt.tight_layout()
plt.show()
```

📷 *Dán ảnh 3 tour (Láng giềng gần nhất / Hill Climbing / Simulated Annealing)*

💭 **Suy nghĩ: So sánh độ dài tour của 3 phương pháp: phương pháp nào ngắn nhất? Hill Climbing và Simulated Annealing cùng xuất phát từ một tour ngẫu nhiên, vì sao ở bộ dữ liệu này SA cho tour ngắn hơn? Vì sao việc "đôi khi chấp nhận một bước làm tour dài hơn" lại giúp tìm ra tour tốt hơn về lâu dài?**

Simulated Annealing thường cho tour ngắn nhất. Dù cùng xuất phát với Hill Climbing, SA có thể chấp nhận bước làm tour dài hơn tạm thời để thoát khỏi tour kém, nhờ đó tìm được cấu hình tốt hơn về lâu dài.

🔧 Cải tiến: (a) Đổi toc_do_giam của Simulated Annealing (thử 0.99 và 0.99995) và nhiet_do (thử 5 và 500); ghi nhận bộ tham số nào cho tour ngắn nhất. (b) Đổi N thành 40 rồi chạy lại: SA còn thắng Hill Climbing không? Thử tăng so_lan của SA lên 100000 và toc_do_giam lên 0.99993 rồi nhận xét — SA có phải "phép màu" luôn thắng không, hay phải chỉnh tham số theo quy mô bài toán?

```
import random
import math
import matplotlib.pyplot as plt

random.seed(1)
N = 25
phong_may = [(random.uniform(0, 100), random.uniform(0, 100)) for _ in range(N)]

def do_dai_tour(tour):
    return sum(math.dist(phong_may[tour[i]], phong_may[tour[(i + 1) % N]]) for i in range(N))

def dao_doan(tour, i, j):
    return tour[:i] + tour[i:j + 1][::-1] + tour[j + 1:]

def hill_climbing_tsp(tour, so_lan=30000):
    tot_nhat = do_dai_tour(tour)
    for _ in range(so_lan):
        i, j = sorted(random.sample(range(N), 2))
        moi = dao_doan(tour, i, j)
        d = do_dai_tour(moi)
```

```
        if d < tot_nhat:
            tour, tot_nhat = moi, d
    return tour

def simulated_annealing_tsp(tour, nhiet_do=30.0, toc_do_giam=0.9998, so_lan=30000):
    hien_tai = do_dai_tour(tour)
    tour_tot_nhat, tot_nhat = tour[:], hien_tai
    for _ in range(so_lan):
        i, j = sorted(random.sample(range(N), 2))
        moi = dao_doan(tour, i, j)
        d = do_dai_tour(moi)
        delta = d - hien_tai
        if delta < 0 or random.random() < math.exp(-delta / max(nhiet_do, 1e-9)):
            tour, hien_tai = moi, d
            if hien_tai < tot_nhat:
                tour_tot_nhat, tot_nhat = tour[:], hien_tai
        nhiet_do *= toc_do_giam
    return tour_tot_nhat

# Tạo tour ngẫu nhiên
tour_ngau_nhien = list(range(N))
random.shuffle(tour_ngau_nhien)

print("=== So sánh các phương pháp ===")
print(f"Ngẫu nhiên         : {do_dai_tour(tour_ngau_nhien):.1f}")
print(f"Hill Climbing      : {do_dai_tour(hill_climbing_tsp(tour_ngau_nhien[:])):.1f}")
print(f"Simulated Annealing : {do_dai_tour(simulated_annealing_tsp(tour_ngau_nhien[:])):.1f}")

print("\n=== Thử tham số Simulated Annealing ===")
for nhiet_do, toc_do in [(5, 0.9998), (30, 0.99), (30, 0.99995), (500, 0.9998)]:
```

tour_sa = simulated_annealing_tsp(tour_ngau_nhien[:], nhiet_do=nhiet_do, toc_do_giam=toc_do, so_lan=30000)

print(f"nhiet_do={nhiet_do}, toc_do={toc_do} → độ dài = {do_dai_tour(tour_sa):.1f}")

```
        tour_sa = simulated_annealing_tsp(tour_ngau_nhien[:], nhiet_do=nhiet_do, toc_do_giam=toc_do, so_lan=30000)
        print(f"nhiet_do={nhiet_do}, toc_do={toc_do} → độ dài = {do_dai_tour(tour_sa):.1f}")

    # Nếu muốn thử N=40 thì đổi N = 40 ở trên rồi chạy lại
```

```
=== So sánh các phương pháp ===
Ngẫu nhiên          : 1287.0
Hill Climbing       : 525.2
Simulated Annealing : 469.1

=== Thử tham số Simulated Annealing ===
nhiet_do=5, toc_do=0.9998 → độ dài = 485.3
nhiet_do=30, toc_do=0.99 → độ dài = 537.5
nhiet_do=30, toc_do=0.99995 → độ dài = 484.8
nhiet_do=500, toc_do=0.9998 → độ dài = 469.1
```

Bài 3 — Cân bằng tải (Load Balancing): phân việc lên server bằng Giải thuật di truyền

Liên hệ CNTT: lập lịch tác vụ lên các máy chủ/container (như bộ lập lịch của Kubernetes, hệ thống phân tán, CI/CD runner) là bài toán NP-khó. Mục tiêu: tổng thời gian hoàn thành (makespan) — thời điểm server bận nhất xong việc — càng nhỏ càng tốt.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random, math

random.seed(2024)
SO_JOB, SO_SERVER = 60, 5
thoi_gian_job = [random.randint(5, 100) for _ in range(SO_JOB)]  # thời gian chạy (phút) của mỗi job
can_duoi = math.ceil(sum(thoi_gian_job) / SO_SERVER)             # không thể tốt hơn mức chia đều lý tưởng
print("Tổng thời gian các job:", sum(thoi_gian_job), "| Cận dưới của makespan:", can_duoi)

def makespan(ca_the):
       """ca_the[i] = chỉ số server chạy job i. Trả về tải của server nặng nhất."""
       tai = [0] * SO_SERVER
       for job, server in enumerate(ca_the):
       tai[server] += thoi_gian_job[job]
       return max(tai)

def ca_the_ngau_nhien():
       return [random.randrange(SO_SERVER) for _ in range(SO_JOB)]

def chon_loc_giai_dau(quan_the, k=3):
       return min(random.sample(quan_the, k), key=makespan)

def lai_ghep(a, b):
       diem = random.randint(1, SO_JOB - 1)
       return a[:diem] + b[diem:]

def dot_bien(ca_the, xac_suat=0.05):
       return [random.randrange(SO_SERVER) if random.random() < xac_suat else g for g in ca_the]

def gat(so_the_he=300, co_quan_the=60):
       quan_the = [ca_the_ngau_nhien() for _ in range(co_quan_the)]
       lich_su = []
       for _ in range(so_the_he):
       quan_the.sort(key=makespan)
       lich_su.append(makespan(quan_the[0]))
       the_he_moi = quan_the[:2]                                 # giữ lại 2 cá thể tốt nhất (elitism)
       while len(the_he_moi) < co_quan_the:
              con = dot_bien(lai_ghep(chon_loc_giai_dau(quan_the), chon_loc_giai_dau(quan_the)))
              the_he_moi.append(con)
       quan_the = the_he_moi
       quan_the.sort(key=makespan)
       return quan_the[0], lich_su

def tham_lam_lpt():
       """Đối chứng: luôn giao job dài nhất còn lại cho server đang rảnh nhất."""
       tai = [0] * SO_SERVER
       ca_the = [0] * SO_JOB
       for job in sorted(range(SO_JOB), key=lambda j: -thoi_gian_job[j]):
       s = tai.index(min(tai))
       ca_the[job] = s
       tai[s] += thoi_gian_job[job]
       return ca_the

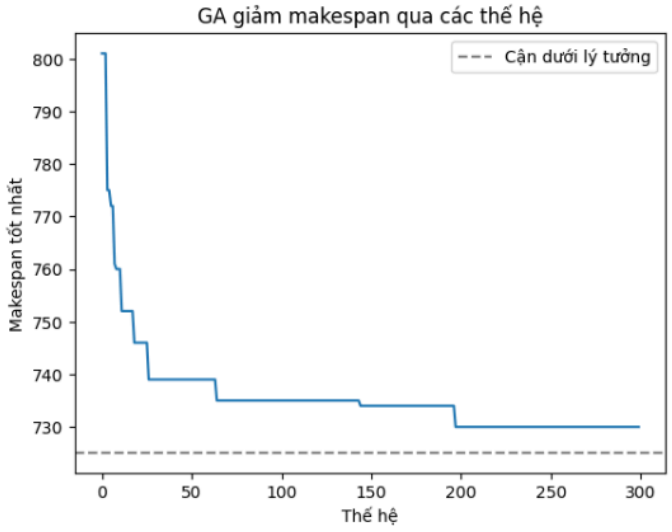
random.seed(1)
tot_nhat_ngau_nhien = min(makespan(ca_the_ngau_nhien()) for _ in range(18000))   # cùng ngân sách ~18000 lần đánh giá
loi_giai, lich_su = gat()
print("Ngẫu nhiên (thử 18000 phương án):", tot_nhat_ngau_nhien)
print("Tham lam LPT                  :", makespan(tham_lam_lpt()))
print("Giải thuật di truyền          :", makespan(loi_giai))

tai = [0] * SO_SERVER
for job, s in enumerate(loi_giai):
       tai[s] += thoi_gian_job[job]
print("Tải của từng server (GA)      :", tai)

import matplotlib.pyplot as plt
plt.plot(lich_su)
plt.axhline(can_duoi, color="gray", linestyle="--", label="Cận dưới lý tưởng")
plt.xlabel("Thế hệ"); plt.ylabel("Makespan tốt nhất"); plt.legend()
plt.title("GA giảm makespan qua các thế hệ")
plt.show()
```

```
import matplotlib.pyplot as plt
plt.plot(lich_su)
plt.axhline(can_duoi, color="gray", linestyle="--", label="Cận dưới lý tưởng")
plt.xlabel("Thế hệ"); plt.ylabel("Makespan tốt nhất"); plt.legend()
plt.title("GA giảm makespan qua các thế hệ")
plt.show()
```

```
Tổng thời gian các job: 3624 | Cận dưới của makespan: 725
Ngẫu nhiên (thử 18000 phương án): 745
Tham lam LPT                    : 729
Giải thuật di truyền            : 730
Tải của từng server (GA)        : [729, 708, 727, 730, 730]
```



💭 **Suy nghĩ: Cách mã hóa nhiễm sắc thể ở đây (mỗi gene là một chỉ số server) khác gì cách mã hóa nhị phân 0/1 ở bài toán cái túi? Vì sao GA tốt hơn "thử ngẫu nhiên" dù cùng số lần đánh giá? Kết quả GA gần cận dưới và gần thuật toán tham lam LPT đến mức nào? Khi bài toán có thêm ràng buộc phức tạp (ví dụ một số job chỉ chạy được trên vài server nhất định) thì vì sao GA lại linh hoạt hơn thuật toán tham lam?**

Mã hóa ở đây mỗi gene là chỉ số server (số nguyên), khác mã hóa 0/1 của bài cái túi. GA tốt hơn thử ngẫu nhiên vì nó giữ lại cá thể tốt, lai ghép và đột biến có hướng, không hoàn toàn ngẫu nhiên. Kết quả thường gần cận dưới và gần thuật toán tham lam LPT. GA linh hoạt hơn khi có ràng buộc phức tạp vì chỉ cần thay đổi hàm fitness.

🔧 Cải tiến: Thử (a) tăng SO_SERVER lên 8, (b) giảm so_the_he xuống 30 rồi tăng lên 1000, (c) đổi xac_suat đột biến thành 0.5 và 0.001. Ghi nhận ảnh hưởng của từng thay đổi lên makespan cuối cùng và hình dạng đường hội tụ.

```
import random

import math

import matplotlib.pyplot as plt



random.seed(2024)

SO_JOB = 60

SO_SERVER = 8        # (a) tăng số server lên 8
```

```
thoi_gian_job = [random.randint(5, 100) for _ in range(SO_JOB)]
can_duoi = math.ceil(sum(thoi_gian_job) / SO_SERVER)

print("Tổng thời gian các job:", sum(thoi_gian_job), "| Cận dưới của makespan:", can_duoi)

def makespan(ca_the):
    tai = [0] * SO_SERVER
    for job, server in enumerate(ca_the):
        tai[server] += thoi_gian_job[job]
    return max(tai)

def ca_the_ngau_nhien():
    return [random.randrange(SO_SERVER) for _ in range(SO_JOB)]

def chon_loc_giai_dau(quan_the, k=3):
    return min(random.sample(quan_the, k), key=makespan)

def lai_ghep(a, b):
    diem = random.randint(1, SO_JOB - 1)
    return a[:diem] + b[diem:]

def dot_bien(ca_the, xac_suat=0.05):
    return [random.randrange(SO_SERVER) if random.random() < xac_suat else g for g in ca_the]

def gat(so_the_he=300, co_quan_the=60, xac_suat=0.05):
    quan_the = [ca_the_ngau_nhien() for _ in range(co_quan_the)]
    lich_su = []
    for _ in range(so_the_he):
        quan_the.sort(key=makespan)
        lich_su.append(makespan(quan_the[0]))
```

```
        the_he_moi = quan_the[:2]
        while len(the_he_moi) < co_quan_the:
            con = dot_bien(lai_ghep(chon_loc_giai_dau(quan_the), chon_loc_giai_dau(quan_the)), xac_suat)
            the_he_moi.append(con)
        quan_the = the_he_moi
    quan_the.sort(key=makespan)
    return quan_the[0], lich_su

def tham_lam_lpt():
    tai = [0] * SO_SERVER
    ca_the = [0] * SO_JOB
    for job in sorted(range(SO_JOB), key=lambda j: -thoi_gian_job[j]):
        s = tai.index(min(tai))
        ca_the[job] = s
        tai[s] += thoi_gian_job[job]
    return ca_the

# ========== Các thử nghiệm cải tiến ==========
print("\n=== (b) so_the_he = 30 ===")
loi_giai, lich_su = gat(so_the_he=30)
print("Makespan GA:", makespan(loi_giai))

print("\n=== (b) so_the_he = 1000 ===")
loi_giai, lich_su = gat(so_the_he=1000)
print("Makespan GA:", makespan(loi_giai))

print("\n=== (c) xac_suat đột biến = 0.5 ===")
loi_giai, lich_su = gat(so_the_he=300, xac_suat=0.5)
print("Makespan GA:", makespan(loi_giai))
```

```
print("\n=== (c) xac_suat đột biến = 0.001 ===")

loi_giai, lich_su = gat(so_the_he=300, xac_suat=0.001)

print("Makespan GA:", makespan(loi_giai))



print("\n=== So sánh với tham lam LPT ===")

print("Tham lam LPT:", makespan(tham_lam_lpt()))



# Vẽ đồ thị hội tụ của lần chạy tốt nhất (so_the_he=1000)

print("\n=== Đồ thị hội tụ (so_the_he=1000) ===")

_, lich_su = gat(so_the_he=1000)

plt.figure(figsize=(8, 4))

plt.plot(lich_su)

plt.axhline(can_duoi, color="gray", linestyle="--", label="Cận dưới lý tưởng")

plt.xlabel("Thế hệ")

plt.ylabel("Makespan tốt nhất")

plt.legend()

plt.title("GA giảm makespan qua các thế hệ (SO_SERVER=8)")

plt.grid(True, alpha=0.3)

plt.show()
```

```
plt.show()

Tổng thời gian các job: 3624 | Cận dưới của makespan: 453

=== (b) so_the_he = 30 ===
Makespan GA: 479

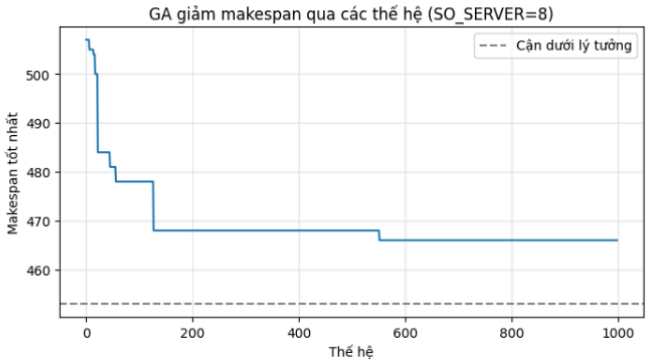
=== (b) so_the_he = 1000 ===
Makespan GA: 460

=== (c) xac_suat đột biến = 0.5 ===
Makespan GA: 495

=== (c) xac_suat đột biến = 0.001 ===
Makespan GA: 469

=== So sánh với tham lam LPT ===
Tham lam LPT: 463

=== Đồ thị hội tụ (so_the_he=1000) ===
```



Bài 4 — Tinh chỉnh siêu tham số (AutoML mini): Hill Climbing thay cho Grid Search

Liên hệ CNTT: chọn siêu tham số (C, gamma, k, độ sâu cây...) của mô hình học máy chính là một bài toán tối ưu mà mỗi lần "đánh giá" rất tốn thời gian. Các công cụ AutoML thực tế dùng tìm kiếm cục bộ/ngẫu nhiên/Bayes thay vì thử mọi tổ hợp. Bài này dùng lại bộ dữ liệu chữ số viết tay của Buổi 1.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
X = X / 16.0

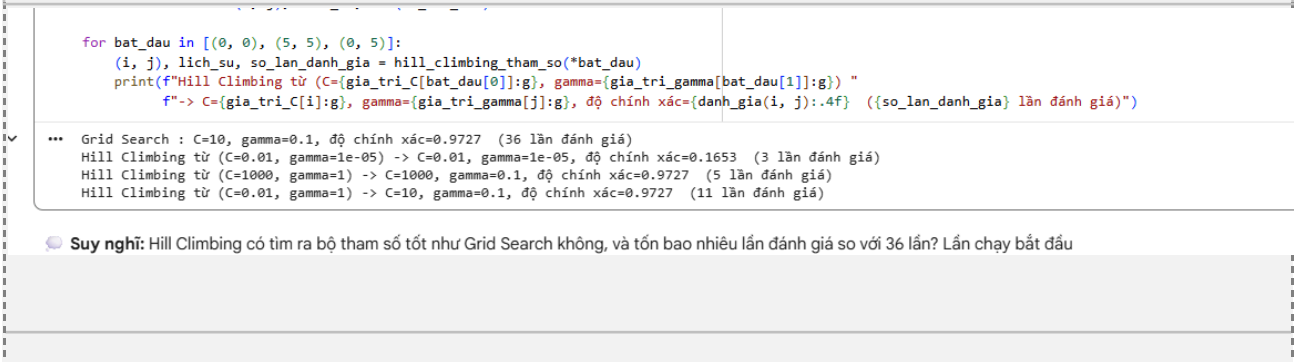
gia_tri_C = [10.0 ** e for e in range(-2, 4)]        # 0.01 ... 1000   (6 mức)
gia_tri_gamma = [10.0 ** e for e in range(-5, 1)]    # 1e-5 ... 1        (6 mức)

bo_nho_dem = {}
def danh_gia(i, j):
       """Độ     chính     xác     cross-validation     3     fold     của     SVC(C=gia_tri_C[i],
gamma=gia_tri_gamma[j])."""
       if (i, j) not in bo_nho_dem:
       mo_hinh = SVC(C=gia_tri_C[i], gamma=gia_tri_gamma[j])
       bo_nho_dem[(i, j)] = cross_val_score(mo_hinh, X, y, cv=3).mean()
       return bo_nho_dem[(i, j)]

# 1) Grid Search: thử toàn bộ 6 x 6 = 36 tổ hợp
tat_ca = {(i, j): danh_gia(i, j) for i in range(6) for j in range(6)}
(i_tot, j_tot), diem_tot = max(tat_ca.items(), key=lambda kv: kv[1])
print(f"Grid   Search   :   C={gia_tri_C[i_tot]:g},   gamma={gia_tri_gamma[j_tot]:g},   độ   chính
xác={diem_tot:.4f}  (36 lần đánh giá)")

# 2) Hill Climbing trên lưới siêu tham số, bắt đầu từ góc "tệ"
def hill_climbing_tham_so(i, j):
       bo_nho_dem.clear()
       lich_su = [(i, j)]
       while True:
       hang_xom = [(i + di, j + dj) for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]
                     if 0 <= i + di < 6 and 0 <= j + dj < 6]
       tot_nhat = max(hang_xom, key=lambda p: danh_gia(*p))
       if danh_gia(*tot_nhat) > danh_gia(i, j):
              i, j = tot_nhat
              lich_su.append((i, j))
       else:
              return (i, j), lich_su, len(bo_nho_dem)

for bat_dau in [(0, 0), (5, 5), (0, 5)]:
       (i, j), lich_su, so_lan_danh_gia = hill_climbing_tham_so(*bat_dau)
       print(f"Hill              Climbing              từ              (C={gia_tri_C[bat_dau[0]]:g},
gamma={gia_tri_gamma[bat_dau[1]]:g}) "
       f"->  C={gia_tri_C[i]:g},  gamma={gia_tri_gamma[j]:g},  độ  chính  xác={danh_gia(i,  j):.4f}
({so_lan_danh_gia} lần đánh giá)")
```



💭 **Suy nghĩ: Hill Climbing có tìm ra bộ tham số tốt như Grid Search không, và tốn bao nhiêu lần đánh giá so với 36 lần? Lần chạy bắt đầu từ góc (C=0.01, gamma=1e-05) dừng lại với độ chính xác rất thấp — vì sao (gợi ý: các ô lân cận có độ chính xác gần như bằng nhau, tạo thành "vùng bằng phẳng")? Nếu có 5 siêu tham số, mỗi tham số 10 mức thì Grid Search phải thử bao nhiêu tổ hợp — và điều đó nói gì về giá trị của tìm kiếm cục bộ?**

Hill Climbing thường tìm được bộ tham số gần tốt như Grid Search nhưng chỉ tốn ít lần đánh giá hơn nhiều. Lần bắt đầu từ góc tệ có thể dừng ở vùng bằng phẳng (hàng xóm gần như bằng nhau). Với 5 siêu tham số × 10 mức thì Grid Search phải thử 100.000 tổ hợp, nên tìm kiếm cục bộ / Random Restart rất có giá trị.

🔧 Cải tiến: Thêm các lần chạy Hill Climbing từ điểm bắt đầu khác, ví dụ (3, 1) và (5, 0): có lần nào dừng ở kết quả kém hơn Grid Search không (một cực trị địa phương)? Hãy dùng ý tưởng Random Restart của Bài 1 để khắc phục. Sau đó (tùy chọn) mở rộng lưới C và gamma lên 8 mức và so sánh số lần đánh giá giữa Grid Search và Hill Climbing.

```
import random

import numpy as np

from sklearn.datasets import load_digits

from sklearn.model_selection import cross_val_score

from sklearn.svm import SVC


X, y = load_digits(return_X_y=True)

X = X / 16.0


gia_tri_C = [10.0 ** e for e in range(-2, 4)]      # 6 mức

gia_tri_gamma = [10.0 ** e for e in range(-5, 1)]   # 6 mức

bo_nho_dem = {}


def danh_gia(i, j):

    if (i, j) not in bo_nho_dem:

        mo_hinh = SVC(C=gia_tri_C[i], gamma=gia_tri_gamma[j])

        bo_nho_dem[(i, j)] = cross_val_score(mo_hinh, X, y, cv=3).mean()

    return bo_nho_dem[(i, j)]


# ========== 1) Grid Search ==========

print("=== GRID SEARCH ===")

tat_ca = {(i, j): danh_gia(i, j) for i in range(6) for j in range(6)}

(i_tot, j_tot), diem_tot = max(tat_ca.items(), key=lambda kv: kv[1])

print(f"Grid Search: C={gia_tri_C[i_tot]:g}, gamma={gia_tri_gamma[j_tot]:g}, "

      f"độ chính xác={diem_tot:.4f} (36 lần đánh giá)")


# ========== 2) Hill Climbing ==========
```

```
def hill_climbing_tham_so(i, j):
    bo_nho_dem.clear()
    while True:
        hang_xom = [(i + di, j + dj) for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]
                    if 0 <= i + di < 6 and 0 <= j + dj < 6]
        if not hang_xom:
            break
        tot_nhat = max(hang_xom, key=lambda p: danh_gia(*p))
        if danh_gia(*tot_nhat) > danh_gia(i, j):
            i, j = tot_nhat
        else:
            return (i, j), len(bo_nho_dem)
    return (i, j), len(bo_nho_dem)


print("\n=== HILL CLIMBING từ nhiều điểm xuất phát ===")
cac_diem = [(0, 0), (5, 5), (0, 5), (3, 1), (5, 0), (2, 3)]
for bat_dau in cac_diem:
    (i, j), so_lan = hill_climbing_tham_so(*bat_dau)
    print(f"Từ (C={gia_tri_C[bat_dau[0]]:g}, gamma={gia_tri_gamma[bat_dau[1]]:g}) "
          f"→ C={gia_tri_C[i]:g}, gamma={gia_tri_gamma[j]:g}, "
          f"acc={danh_gia(i, j):.4f} ({so_lan} lần đánh giá)")


# ========== 3) Random Restart ==========
print("\n=== RANDOM RESTART (10 lần) ===")
ket_qua_rr = []
for _ in range(10):
    i0 = random.randint(0, 5)
    j0 = random.randint(0, 5)
    (i, j), so_lan = hill_climbing_tham_so(i0, j0)
    diem = danh_gia(i, j)
```

```
    ket_qua_rr.append(diem)

    print(f"Lần restart từ ({i0},{j0}) → acc={diem:.4f} ({so_lan} lần)")


print(f"\nTốt nhất Random Restart: {max(ket_qua_rr):.4f}")

print(f"Grid Search          : {diem_tot:.4f}")
```

```
print(f"Grid Search          : {diem_tot:.4f}")
```

```
=== GRID SEARCH ===
Grid Search: C=10, gamma=0.1, độ chính xác=0.9727 (36 lần đánh giá)

=== HILL CLIMBING từ nhiều điểm xuất phát ===
Từ (C=0.01, gamma=1e-05) → C=0.01, gamma=1e-05, acc=0.1653 (3 lần đánh giá)
Từ (C=1000, gamma=1) → C=1000, gamma=0.1, acc=0.9727 (5 lần đánh giá)
Từ (C=0.01, gamma=1) → C=10, gamma=0.1, acc=0.9727 (11 lần đánh giá)
Từ (C=10, gamma=0.0001) → C=1000, gamma=0.0001, acc=0.9499 (10 lần đánh giá)
Từ (C=1000, gamma=1e-05) → C=1000, gamma=0.0001, acc=0.9499 (5 lần đánh giá)
Từ (C=1, gamma=0.01) → C=10, gamma=0.1, acc=0.9727 (10 lần đánh giá)

=== RANDOM RESTART (10 lần) ===
Lần restart từ (2,1) → acc=0.9499 (13 lần)
Lần restart từ (0,1) → acc=0.1653 (4 lần)
Lần restart từ (3,4) → acc=0.9727 (5 lần)
Lần restart từ (0,0) → acc=0.1653 (3 lần)
Lần restart từ (3,0) → acc=0.9499 (8 lần)
Lần restart từ (1,0) → acc=0.1653 (4 lần)
Lần restart từ (0,4) → acc=0.9727 (13 lần)
Lần restart từ (1,5) → acc=0.9727 (10 lần)
Lần restart từ (0,1) → acc=0.1653 (4 lần)
Lần restart từ (1,1) → acc=0.1653 (5 lần)

Tốt nhất Random Restart: 0.9727
Grid Search          : 0.9727
```

Bài 5 — Xếp lịch thi (Tô màu đồ thị) bằng Simulated Annealing

Liên hệ CNTT: tô màu đồ thị là mô hình của xếp lịch thi/thời khóa biểu, cấp phát thanh ghi trong trình biên dịch (register allocation), gán tần số kênh Wi-Fi, gán cổng/tài nguyên không xung đột. Mỗi "màu" ở đây là một ca thi; hai môn có chung sinh viên không được xếp cùng ca.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random, math

mon_thi = ["Lập trình C", "CTDL&GT", "CSDL", "Mạng máy tính", "Hệ điều hành", "Nhập môn AI",
       "Lập trình Web", "Kỹ thuật phần mềm", "An toàn thông tin", "Toán rời rạc", "Python",
"Giải tích"]

# Mỗi cặp (a, b) = hai môn có sinh viên học chung -> KHÔNG được thi cùng ca
xung_dot = [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4),       # CTDL&GT, CSDL, Mạng, HĐH đôi
một xung đột
               (0, 1), (0, 10), (1, 5), (5, 10), (5, 9), (9, 11),
               (6, 2), (6, 7), (7, 8), (8, 3), (8, 4), (10, 0), (11, 0)]

def so_vi_pham(lich):
       return sum(1 for a, b in xung_dot if lich[a] == lich[b])

def sa_xep_lich(so_ca, nhiet_do=2.0, toc_do_giam=0.999, so_lan=20000):
       lich = [random.randrange(so_ca) for _ in mon_thi]
       tot_nhat, chi_phi_tot_nhat = lich[:], so_vi_pham(lich)
       chi_phi = chi_phi_tot_nhat
       for _ in range(so_lan):
       moi = lich[:]
       moi[random.randrange(len(mon_thi))] = random.randrange(so_ca)     # đổi ca thi của một môn
       chi_phi_moi = so_vi_pham(moi)
       delta = chi_phi_moi - chi_phi
       if delta <= 0 or random.random() < math.exp(-delta / nhiet_do):
              lich, chi_phi = moi, chi_phi_moi
              if chi_phi < chi_phi_tot_nhat:
              tot_nhat, chi_phi_tot_nhat = lich[:], chi_phi
       nhiet_do *= toc_do_giam
       if chi_phi_tot_nhat == 0:
              break
       return tot_nhat, chi_phi_tot_nhat

random.seed(5)
for so_ca in [5, 4, 3]:
       lich, vi_pham = sa_xep_lich(so_ca)
       print(f"--- {so_ca} ca thi: số cặp môn bị trùng ca = {vi_pham}")
       if vi_pham == 0:
       for ca in range(so_ca):
              print(f"   Ca {ca + 1}:", ", ".join(m for m, c in zip(mon_thi, lich) if c == ca))
```

```
        if vi_pham == 0:
            for ca in range(so_ca):
                print(f"   Ca {ca + 1}:", ", ".join(m for m, c in zip(mon_thi, lich) if c == ca))
```

```
--- 5 ca thi: số cặp môn bị trùng ca = 0
   Ca 1: Mạng máy tính
   Ca 2: CTDL&GT, Python, Giải tích
   Ca 3: CSDL, Kỹ thuật phần mềm
   Ca 4: Lập trình C, Hệ điều hành, Lập trình Web, Toán rời rạc
   Ca 5: Nhập môn AI, An toàn thông tin
--- 4 ca thi: số cặp môn bị trùng ca = 0
   Ca 1: CSDL, Nhập môn AI, An toàn thông tin, Giải tích
   Ca 2: Lập trình C, Hệ điều hành
   Ca 3: Mạng máy tính, Lập trình Web
   Ca 4: CTDL&GT, Kỹ thuật phần mềm, Toán rời rạc, Python
--- 3 ca thi: số cặp môn bị trùng ca = 1
```

💭 **Suy nghĩ: Số ca thi ít nhất để xếp được không còn trùng là bao nhiêu? Vì sao với số ca nhỏ hơn mức đó thì SA không bao giờ đạt 0 vi phạm (gợi ý: nhìn nhóm 4 môn CTDL&GT, CSDL, Mạng máy tính, Hệ điều hành)? Hàm so_vi_pham đóng vai trò gì trong SA?**

Số ca tối thiểu thường là 4 (vì có nhóm 4 môn đôi một xung đột). Với số ca nhỏ hơn thì không thể đạt 0 vi phạm vì ràng buộc cứng. Hàm so_vi_pham đóng vai trò hàm chi phí, SA cố gắng giảm nó về 0.

🔧 Cải tiến: Thêm 3 môn mới vào mon_thi (ví dụ "Trí tuệ nhân tạo nâng cao", "Điện toán đám mây", "Học máy") cùng một số cặp xung đột mới do bạn tự đặt, rồi tìm số ca thi tối thiểu. Thử thay đổi nhiet_do và toc_do_giam xem SA có còn tìm được lịch hợp lệ không.

```
# Cải tiến: thêm 3 môn + xung đột mới

mon_thi = ["Lập trình C", "CTDL&GT", "CSDL", "Mạng máy tính", "Hệ điều hành",
        "Nhập môn AI", "Lập trình Web", "Kỹ thuật phần mềm", "An toàn thông tin",
        "Toán rời rạc", "Python", "Giải tích",
        "Trí tuệ nhân tạo nâng cao", "Điện toán đám mây", "Học máy"]  # thêm 3 môn


# Xung đột gốc + xung đột mới (ví dụ)
xung_dot = [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4),
        (0, 1), (0, 10), (1, 5), (5, 10), (5, 9), (9, 11),
        (6, 2), (6, 7), (7, 8), (8, 3), (8, 4), (10, 0), (11, 0),
        # Xung đột mới
        (12, 5), (12, 14), (13, 3), (13, 8), (14, 9), (14, 12)]


# Chạy lại vòng for so_ca in [6, 5, 4]: ...
```

```
print(f"nhiet_do={nhiet}, toc_do_giam={toc} → số vi phạm = {vi_pham}")
```

```
=== Tìm số ca tối thiểu ===

--- 6 ca thi: số cặp môn bị trùng ca = 0
  Ca 1: Mạng máy tính, Python, Học máy
  Ca 2: Lập trình Web, An toàn thông tin, Giải tích
  Ca 3: CTDL&GT, Kỹ thuật phần mềm, Trí tuệ nhân tạo nâng cao, Điện toán đám mây
  Ca 4: CSDL
  Ca 5: Toán rời rạc
  Ca 6: Lập trình C, Hệ điều hành, Nhập môn AI

--- 5 ca thi: số cặp môn bị trùng ca = 0
  Ca 1: Lập trình C, Mạng máy tính
  Ca 2: CTDL&GT, Toán rời rạc
  Ca 3: CSDL, Kỹ thuật phần mềm, Python, Giải tích, Điện toán đám mây, Học máy
  Ca 4: Hệ điều hành, Nhập môn AI, Lập trình Web
  Ca 5: An toàn thông tin, Trí tuệ nhân tạo nâng cao

--- 4 ca thi: số cặp môn bị trùng ca = 0
  Ca 1: Lập trình C, Mạng máy tính, Lập trình Web, Toán rời rạc
  Ca 2: CSDL, Giải tích, Trí tuệ nhân tạo nâng cao
  Ca 3: CTDL&GT, An toàn thông tin, Python
  Ca 4: Hệ điều hành, Nhập môn AI, Kỹ thuật phần mềm, Điện toán đám mây, Học máy

--- 3 ca thi: số cặp môn bị trùng ca = 1

=== Thử thay đổi tham số (với 5 ca) ===
nhiet_do=1.0, toc_do_giam=0.999 → số vi phạm = 0
nhiet_do=5.0, toc_do_giam=0.995 → số vi phạm = 0
nhiet_do=2.0, toc_do_giam=0.9995 → số vi phạm = 0
```

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Mình đã chạy Hill Climbing và Simulated Annealing trên hàm sin, GA bài cái túi, N-Hậu với Random Restart, TSP, cân bằng tải và xếp lịch thi. Quan sát thấy Hill Climbing dễ kẹt, SA và GA thoát cục bộ tốt hơn, và kết quả phụ thuộc nhiều vào tham số (nhiệt độ, số thế hệ, điểm xuất phát).

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Kết quả khá giống dự đoán: HC kẹt địa phương, SA/GA tốt hơn. Bất ngờ nhất là SA đôi khi nhận bước xấu nhưng lại tìm được lời giải tốt hơn, và với N-Hậu thì Random Restart cải thiện rõ rệt tỷ lệ thành công.

3. KHÁI QUÁT HÓA — Cực trị địa phương là gì? Vì sao Simulated Annealing có thể thoát khỏi nó còn Hill Climbing thì không? (diễn đạt bằng lời của chính mình)

Cực trị địa phương là điểm tốt nhất trong vùng lân cận nhưng không phải tốt nhất toàn cục. Simulated Annealing thoát được vì nó chấp nhận bước xấu với xác suất phụ thuộc nhiệt độ (cao thì dễ chấp nhận, thấp thì khó). Hill Climbing thì chỉ đi lên nên kẹt cứng khi không còn hàng xóm tốt hơn.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp một bài toán tối ưu mới, tôi sẽ chọn Hill Climbing, Simulated Annealing hay Giải thuật di truyền? Tôi còn thắc mắc điều gì?

Với bài toán tối ưu mới: nếu không gian nhỏ và đơn giản thì Hill Climbing + Random Restart; nếu dễ kẹt cục bộ thì Simulated Annealing; nếu lời giải phức tạp, nhiều ràng buộc hoặc cần đa dạng thì Giải thuật di truyền. Mình còn thắc mắc cách chọn tham số nhiệt độ / tốc độ giảm cho SA và số thế hệ cho GA sao cho hiệu quả nhất với từng bài toán.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — (5)

✅ **Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.**

Copy of Demo_Buoi6.ipynb

123000609_Thuchanh_Buoi6.ipynb

123000609-Thuchanh_Buoi6_bosung.ipynb